

Multistep numerical methods

Botond Koroknai

Abstract

This report investigates the application multistep numerical methods for solving ordinary differential equations through the classical physical model of Newton's law of cooling. Beginning with the derivation of the analytical solution, the study develops several numerical discretization techniques including the explicit Adams–Bashforth 2 (AB2) method, the implicit Adams–Moulton method, and Backward Differentiation Formula (BDF).

The report presents the mathematical derivation of each scheme together with their numerical implementation in Python. Particular attention is given to convergence behavior, truncation error analysis, and computational stability. Numerical experiments demonstrate that the AB2 and BDF2 methods achieve second-order convergence, significantly improving accuracy compared to the first-order explicit Euler method.

Furthermore, the stability properties of explicit and implicit methods are examined in the context of stiff differential equations. The results show that explicit methods become unstable in such cases for large step sizes, while implicit approaches such as Adams–Moulton and BDF2 remain stable and robust. Overall, the study highlights the efficiency, accuracy, and stability advantages of higher-order multistep methods for time-dependent physical systems.

1 Physical Context – Newton’s law of cooling

This report is going to study a real-life physical problem with the help of different multistep numerical methods. For this purpose let us consider a classical model from thermodynamics, the **Newton’s law of cooling**. This model describes how the temperature of a hot object changes over time when it is placed in an environment with constant ambient temperature.

The fundamental physical assumption is that the rate of cooling is proportional to the temperature difference between the object and its surroundings. This assumption can be best described using a differential equation:

$$\frac{dT}{dt} = -k(T - T_{\text{room}}) \quad (1)$$

where $T(t)$ denotes the temperature of the object at time t , T_{room} is the constant room temperature, and $k > 0$ is the cooling constant. The negative sign reflects the fact that if the object is hotter than the room, its temperature decreases over time.

For our numerical experiment the following parameter values were chosen: $T_{\text{room}} = 20^\circ \text{C}$, $k = 0.3$, and $T(0) = 100^\circ \text{C}$. This means that initially the object has a temperature of 100°C and gradually cools toward 20°C . Substituting these values into the model gives the specific initial value problem:

$$\frac{dT}{dt} = -0.3(T - 20), \quad T(0) = 100 \quad (2)$$

2 Derivation of the exact solution

Before applying the numerical method, it is useful to derive the exact analytical solution of the differential equation, which will later allow us to compare the numerical approximation with the true physical behavior.

To be able to find this solution we need to solve our differential equation of:

$$\frac{dT}{dt} = -0.3(T - 20) \quad (3)$$

Since this is a first-order linear ordinary differential equation, it can easily be solved by the separation of variables. Once the variables are separated we integrate both sides:

$$\int \frac{1}{T - 20} dT = \int -0.3 dt \quad (4)$$

This yields a result of

$$\ln|T - 20| = -0.3t + C \quad (5)$$

which we exponentiate

$$|T - 20| = e^{-0.3t+C} \quad (6)$$

Using the exponential rule $e^{a+b} = e^a e^b$, we obtain

$$|T - 20| = e^C e^{-0.3t} \quad (7)$$

By renaming the constant e^C as C_2 and absorbing the sign of the absolute value, we find $T - 20 = C_2 e^{-0.3t}$. Therefore, the general solution is:

$$T(t) = 20 + C_2 e^{-0.3t} \quad (8)$$

Now we use the initial condition $T(0) = 100$. Substituting $t = 0$ gives $100 = 20 + C_2 e^0$. Since $e^0 = 1$, we find $100 = 20 + C_2$, which implies $C_2 = 80$. Substituting this back into the general solution yields the exact solution of:

$$T(t) = 20 + 80e^{-0.3t} \quad (9)$$

This solution has a clear physical interpretation:

- The term 20 is the long-term equilibrium temperature.
- The exponential term describes how the excess heat decays over time.
- As $t \rightarrow \infty$, $e^{-0.3t} \rightarrow 0$ so that $T(t) \rightarrow 20$, meaning the object eventually reaches room temperature.

3 Numerical discretisation with Adams–Bashforth 2

Now that we have the exact physical model and its analytical solution, the next step is to construct its numerical approximation using the **two-step Adams–Bashforth method**.

To compute the evolution of the temperature, the AB2 method predicts the value of the ODE at the next time step using the following multi-step formula:

$$T_{n+1} = T_n + \frac{h}{2} (3f_n - f_{n-1}) \quad (10)$$

where $f_n = f(t_n, T_n)$ and $f_{n-1} = f(t_{n-1}, T_{n-1})$. This formula uses the slope information from the two most recent time levels in order to predict the next value. A detailed derivation of this update formula based on linear interpolation is provided in Appendix A.

By substituting the parameters into the function, the numerical update formula for this special case becomes

$$T_{n+1} = T_n + \frac{h}{2} [3(-0.3(T_n - 20)) - (-0.3(T_{n-1} - 20))] \quad (11)$$

A very important practical point is that since this is a two-step method, it requires two starting values: T_0, T_1 . The initial condition provides only $T_0 = 100$, so we must first compute T_1 using a one-step method, for example the explicit Euler method, which has the updated formula of:

$$T_1 = T_0 + h f(t_0, T_0) \quad (12)$$

Once this starting value has been computed, we can proceed with the Adams–Bashforth recursion for all subsequent time steps.

4 AB2 Numerical Implementation

The numerical solution was implemented in Python environment with the help of the `numpy` library for vector operations and `matplotlib` for visualization. The core logic (introduced in equation 10) is encapsulated in the function `solve_ab2`, which handles the initialization with the help of the Forward Euler method followed by the primary Adams–Bashforth recursion.

Listing 1: Python implementation of the AB2 method

```
def solve_ab2(h):
    t = np.arange(0, 10 + h, h)
    T = np.zeros(len(t))

    T[0] = 100 # Initial Condition

    # First step predicted using explicit Euler method
    T[1] = T[0] + h * f(t[0], T[0])

    # AB2 loop
    for n in range(1, len(t) - 1):
        f_n = f(t[n], T[n])
        f_nm1 = f(t[n - 1], T[n - 1])
        T[n + 1] = T[n] + h / 2 * (3 * f_n - f_nm1)

    T_exact = 20 + 80 * np.exp(-0.3 * t)
    max_error = np.max(np.abs(T - T_exact))

    return t, T, T_exact, max_error
```

4.1 Analysis of Results

The implementation produces a discrete set of points representing the object's temperature over time. As it can be seen on the result (Figure 1), the AB2 method tracks the analytical solution $T(t) = 20 + 80e^{-0.3t}$ with high precision.

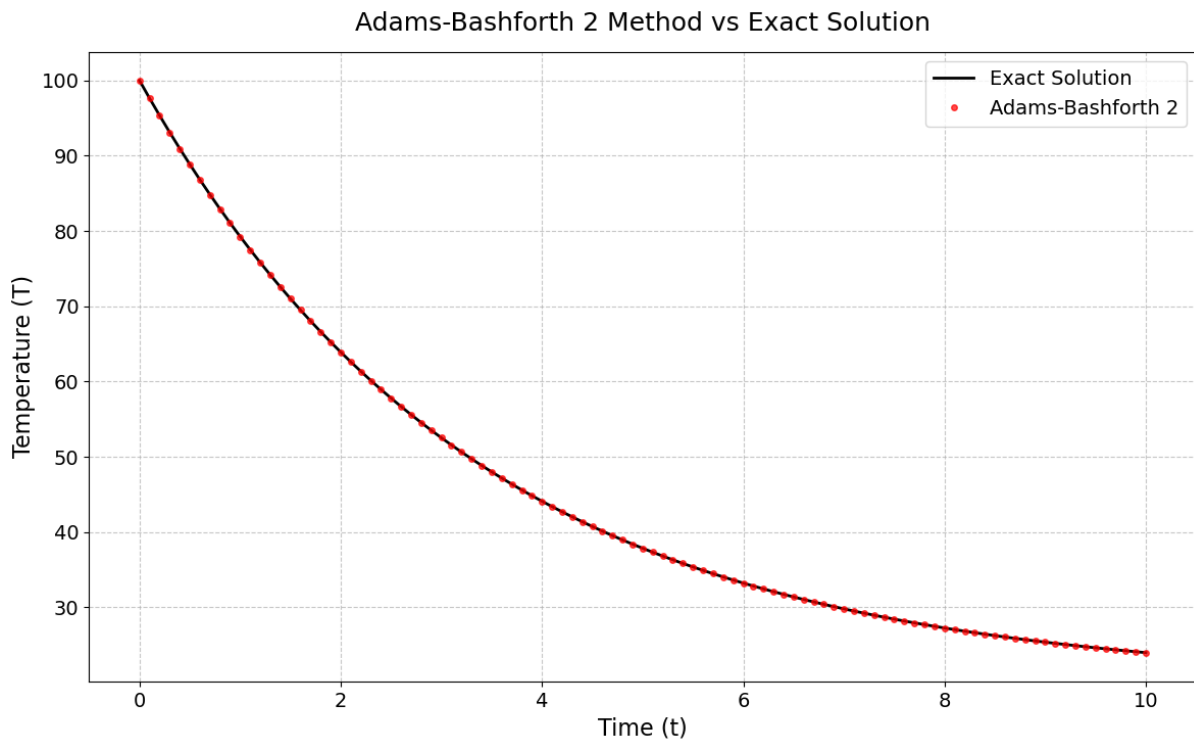


Figure 1: Adams-Bashforth 2 Method vs Exact Solution for $h = 0.1$.

4.2 Error Analysis

To assess the accuracy of the Adams–Bashforth 2 method, the value of absolute error was computed at each time step, defined as:

$$E_n = |T(t_n) - T_n| \quad (13)$$

where $T(t_n)$ is the exact analytical solution and T_n is the numerical approximation. As illustrated in Figure 2, the error initially increases as the local truncation errors accumulate. However, because the system is inherently stable (following an exponential decay toward the ambient temperature), the absolute error eventually levels off and begins to decrease as both the numerical and exact solutions converge toward the steady-state equilibrium of 20°C.

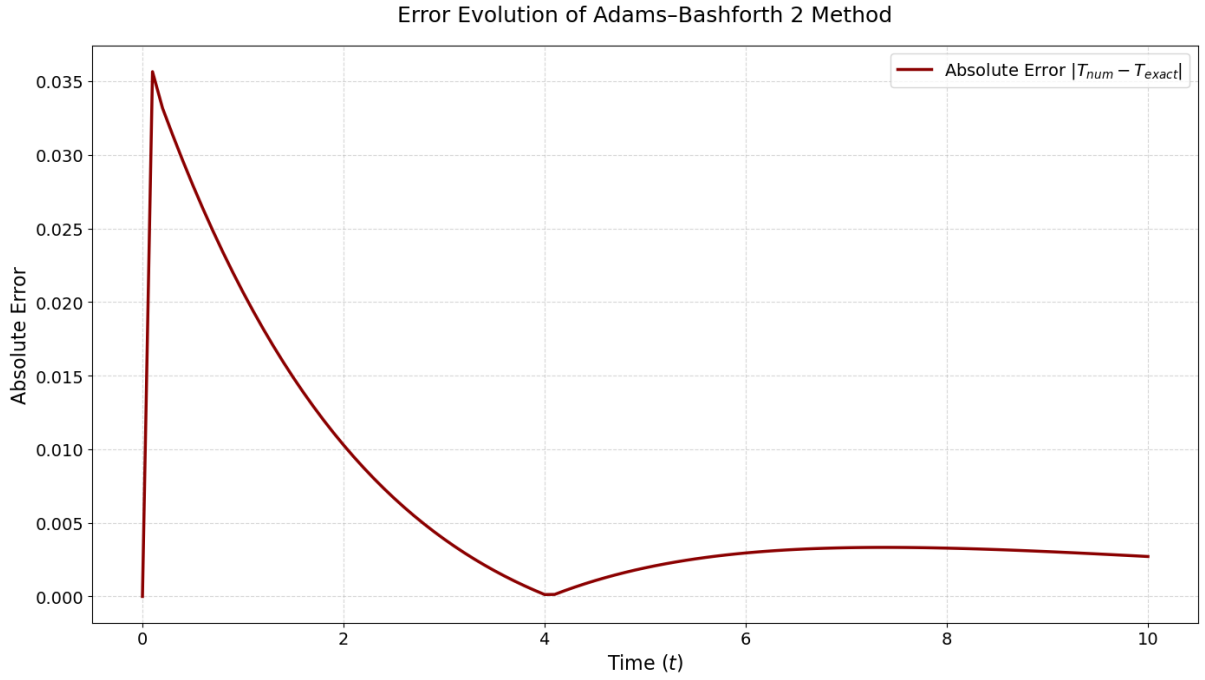


Figure 2: Evolution of the absolute error over time for the AB2 method.

The maximum global error can be used to verify the order of convergence. For the AB2 method, we expect the global error to behave as $O(h^2)$, meaning that halving the step size h should approximately reduce the maximum error by a factor of four. A more comprehensive calculation of the error analysis is provided in Appendix B.

4.3 Convergence Analysis

To verify the accuracy of the Adams–Bashforth 2 method, the behavior of the global truncation error was investigated while the step size h is reduced. For a method of order p , the global error is expected to satisfy the relationship:

$$E(h) \approx Ch^p \quad (14)$$

where C is a constant independent of h . By taking the logarithm of both sides, we obtain $\log(E) \approx p \log(h) + \log(C)$. This implies that on a log-log scale, the error should follow a linear trend with a slope equal to the order of the method.

This theoretical relation can be confirmed by performing a series of simulations using step sizes $h \in \{0.4, 0.2, 0.1, 0.05, 0.025\}$ and recording the maximum absolute error over the entire time interval.

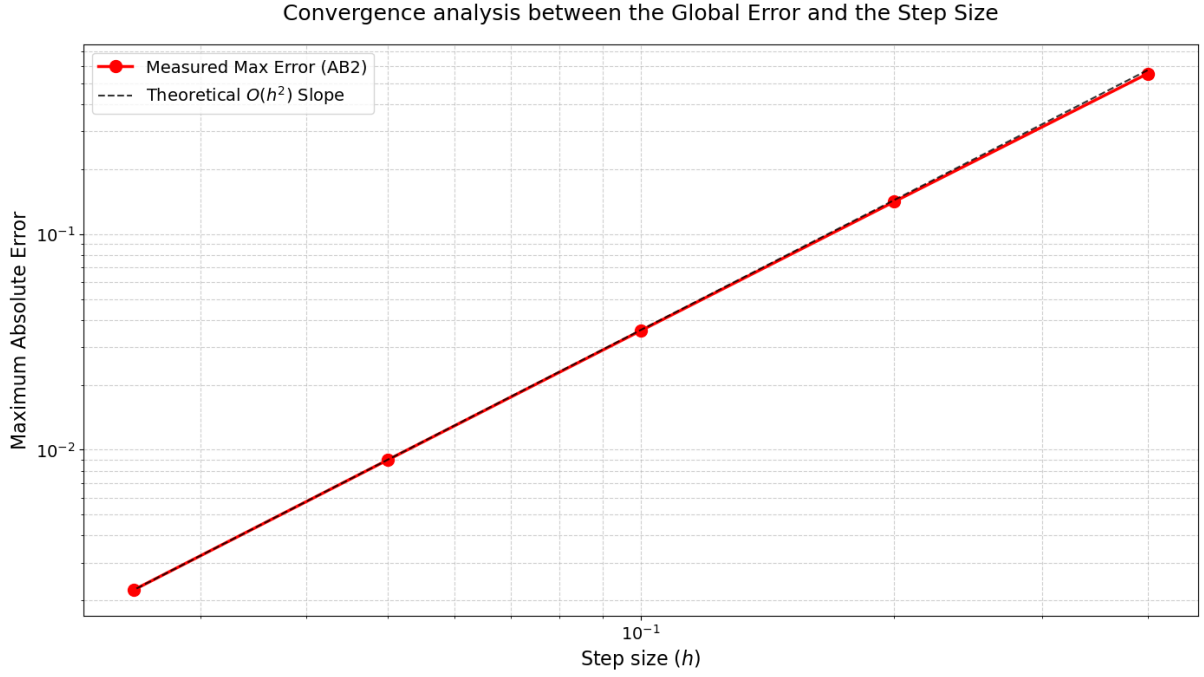


Figure 3: Log-log plot of the maximum global error versus step size h . The observed slope confirms the second-order convergence of the AB2 method.

As shown in Figure 3, the maximum error decreases significantly as the step size becomes smaller. A comparison with a theoretical $O(h^2)$ slope confirms that the Adams–Bashforth 2 method is indeed second-order accurate and it ensures that the numerical implementation is correct and behaves predictably according to the mathematical theory.

4.4 Comparison of Euler and Adams–Bashforth 2

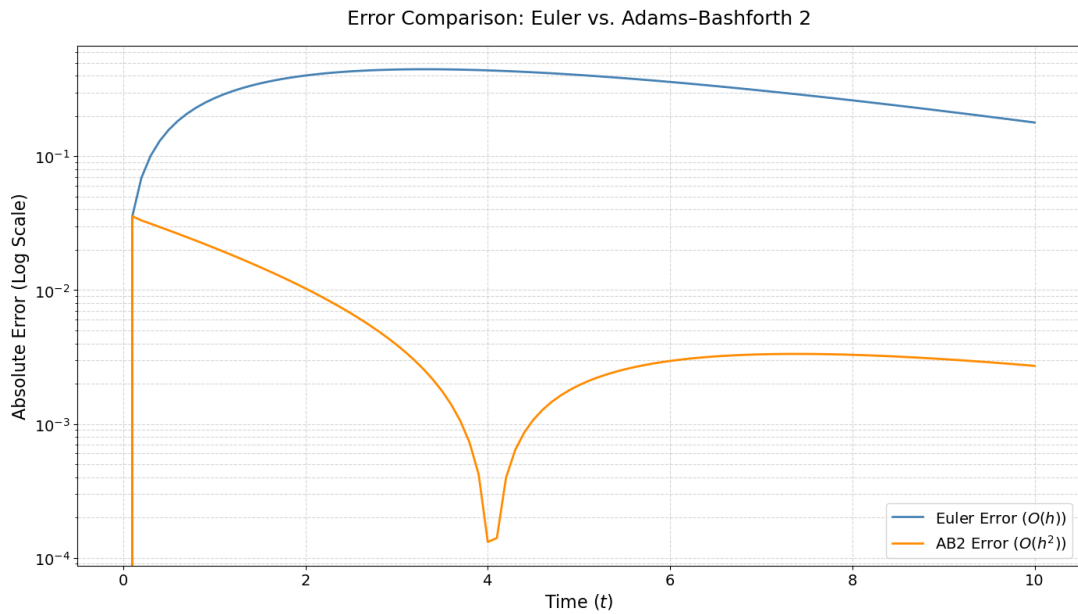


Figure 4: Logarithmic error comparison between the first-order Euler method and the second-order Adams–Bashforth 2 method.

To evaluate the efficiency of the multistep approach, we compare the performance of the first-order explicit Euler method against the second-order Adams–Bashforth 2 (AB2) method using a constant step size of $h = 0.1$. While both methods qualitatively capture the exponential decay toward the ambient temperature, the quantitative disparity in their precision is significant.

The primary advantage of the AB2 method is its ability to achieve higher-order accuracy ($O(h^2)$) without requiring additional evaluations of the derivative function $f(t, T)$ per time step, as it leverages the historical data from the previous point. As illustrated in Figure 4, the logarithmic scale reveals that the AB2 method maintains an absolute error roughly 10 to 100 times smaller than the Euler method.

The "dip" observed in the AB2 error curve near $t \approx 4$ is a characteristic of the log-scale plot, indicating the point where the numerical solution perfectly intersects the exact solution before diverging slightly again. Overall, the comparison confirms that multistep methods offer a superior balance of computational economy and high-fidelity results.

5 Implicit Multistep Methods: Adams–Moulton

To further improve stability and accuracy, we implement an implicit multistep method: the two-step Adams–Moulton (AM2) method. Unlike the Adams–Bashforth family, implicit methods include the unknown future slope $f_{n+1} = f(t_{n+1}, y_{n+1})$ in the update formula. For further details about the origin of the formula read Appendix C

$$T_{n+1} = T_n + \frac{h}{12} (5f_{n+1} + 8f_n - f_{n-1}) \quad (15)$$

Listing 2: Implicit Adams–Moulton 2 implementation with fixed-point iteration

```
def solve_am2(h, tol=1e-10, max_iter=20):
    t = np.arange(0, 10 + h, h)
    T = np.zeros(len(t))
    T[0] = 100
    T[1] = T[0] + h * f(t[0], T[0]) # Euler guess

    for n in range(1, len(t) - 1):
        y_guess = T[n] + h * f(t[n], T[n]) # Predictor
        for _ in range(max_iter):
            f_np1 = f(t[n+1], y_guess)
            y_new = T[n] + (h/12) * (5*f_np1 + 8*f(t[n], T[n]) -
                                     f(t[n-1], T[n-1]))
            if abs(y_new - y_guess) < tol:
                break
            y_guess = y_new # Corrector iteration
        T[n+1] = y_new

    T_exact = 20 + 80 * np.exp(-0.3 * t)
    return t, T, T_exact, np.max(np.abs(T - T_exact))
```

Since T_{n+1} appears on both sides of the equation, it cannot be solved directly for non-linear functions. In this implementation, we solve this algebraic problem using a predictor-corrector approach.

Firstly an initial guess is being generated $T_{n+1}^{(0)}$ using the explicit Euler method (the predictor) and then refine this value through fixed-point iteration (the corrector) until the difference between successive iterations falls below a tolerance of 10^{-10} .

While implicit methods require more computational work per time step due to the iterative solver, they generally offer superior stability properties, especially for "stiff" differential equations where explicit methods might fail or require extremely small step sizes.

6 Global Performance Comparison

A final comparison of the three implemented methods—Explicit Euler, Adams–Bashforth 2 (AB2), and Adams–Moulton (AM)—is presented in Figure 5. By plotting the absolute error on a logarithmic scale, we can observe the distinct "accuracy tiers" created by the order of each method.

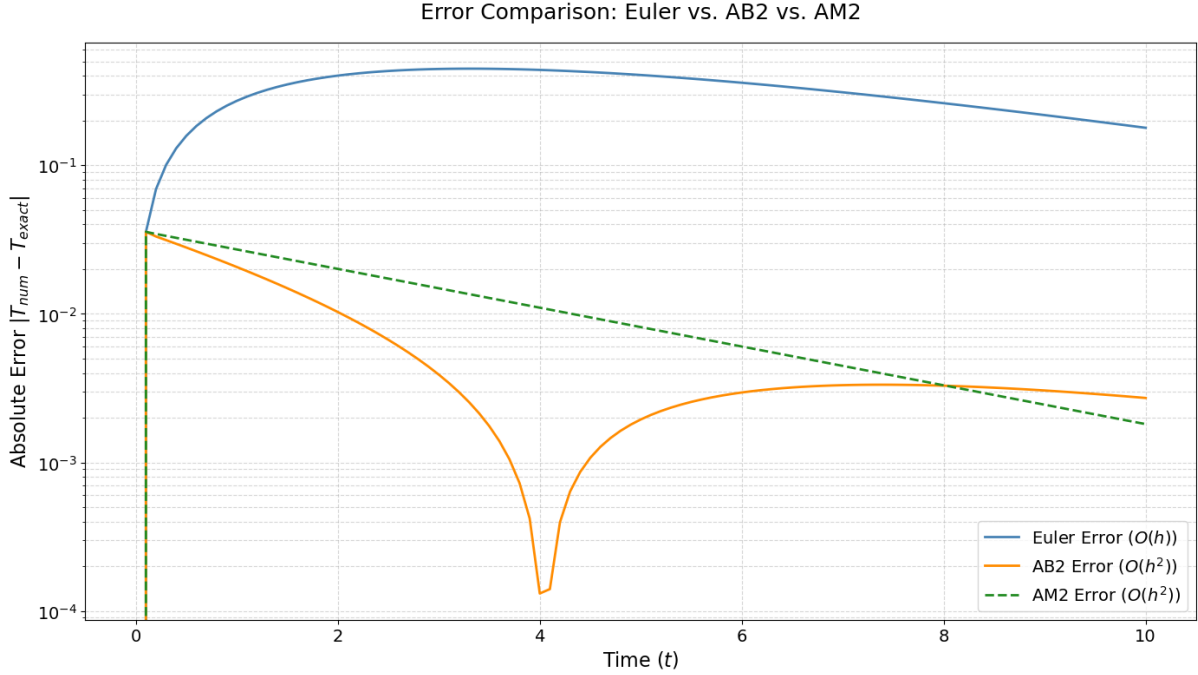


Figure 5: Logarithmic error comparison for all three numerical methods.

7 Implicit Modeling with BDF Methods

In the next part of the report we shift our focus to the Backward Differentiation Formula (BDF). While the physical model of Newton's law of cooling remains unchanged, the method of time discretization introduces a significant conceptual shift.

7.1 The BDF Philosophy

The governing differential equation is:

$$\frac{dT}{dt} = f(t, T) = -k(T - T_{\text{room}}) \quad (16)$$

Unlike explicit methods that project the slope forward from the current point, BDF methods approximate the derivative at the *new* time level t_{n+1} using a reconstruction from past values $(T_{n+1}, T_n, T_{n-1}, \dots)$. We then enforce the physical law at that future point:

$$T'(t_{n+1}) = f(t_{n+1}, T_{n+1}) \quad (17)$$

7.2 BDF1: The Backward Euler Form

The simplest BDF method (BDF1) uses a first-order backward difference:

$$\frac{T_{n+1} - T_n}{h} = -k(T_{n+1} - T_{\text{room}}) \quad (18)$$

This formulation is implicit because the unknown state T_{n+1} appears on both sides of the equation.

Because our cooling model is linear, we can rearrange the BDF1 equation to solve for T_{n+1} directly, bypassing the need for an iterative root-finder. Starting from the discretization:

$$T_{n+1} - T_n = -hkT_{n+1} + hkT_{\text{room}} \quad (19)$$

$$T_{n+1} + hkT_{n+1} = T_n + hkT_{\text{room}} \quad (20)$$

$$T_{n+1}(1 + hk) = T_n + hkT_{\text{room}} \quad (21)$$

The resulting explicit update formula for BDF1 is:

$$T_{n+1} = \frac{T_n + hkT_{\text{room}}}{1 + hk} \quad (22)$$

7.3 Extension to BDF2

To increase the order of accuracy, BDF2 utilizes two historical points to approximate the derivative:

$$\frac{3T_{n+1} - 4T_n + T_{n-1}}{2h} = -k(T_{n+1} - T_{\text{room}}) \quad (23)$$

This method is particularly valuable for long-term simulations where numerical stability is a priority. To see the whole process of deriving the BDF2 formula read Appendix D

7.4 Algebraic Rearrangement for BDF2

To obtain a direct computational update rule, we rearrange the implicit BDF2 equation. Starting by multiplying both sides by $2h$:

$$3T_{n+1} - 4T_n + T_{n-1} = -2hkT_{n+1} + 2hkT_{\text{room}} \quad (24)$$

Next, we collect all terms involving the unknown future state T_{n+1} on the left-hand side:

$$3T_{n+1} + 2hkT_{n+1} = 4T_n - T_{n-1} + 2hkT_{\text{room}} \quad (25)$$

Factoring out T_{n+1} yields:

$$T_{n+1}(3 + 2hk) = 4T_n - T_{n-1} + 2hkT_{\text{room}} \quad (26)$$

Finally, solving for T_{n+1} provides the explicit update formula used in the numerical implementation:

$$T_{n+1} = \frac{4T_n - T_{n-1} + 2hkT_{\text{room}}}{3 + 2hk} \quad (27)$$

7.5 Convergence Analysis of the BDF2 Method

To verify the numerical integrity of the BDF2 implementation, we analyze the behavior of the global truncation error across a descending range of step sizes, from $h = 0.5$ down to $h = 0.01$. Theoretically, a second-order method should satisfy the relationship $E(h) \approx Ch^2$, which appears as a linear trend with a slope of 2 on a log-log scale.

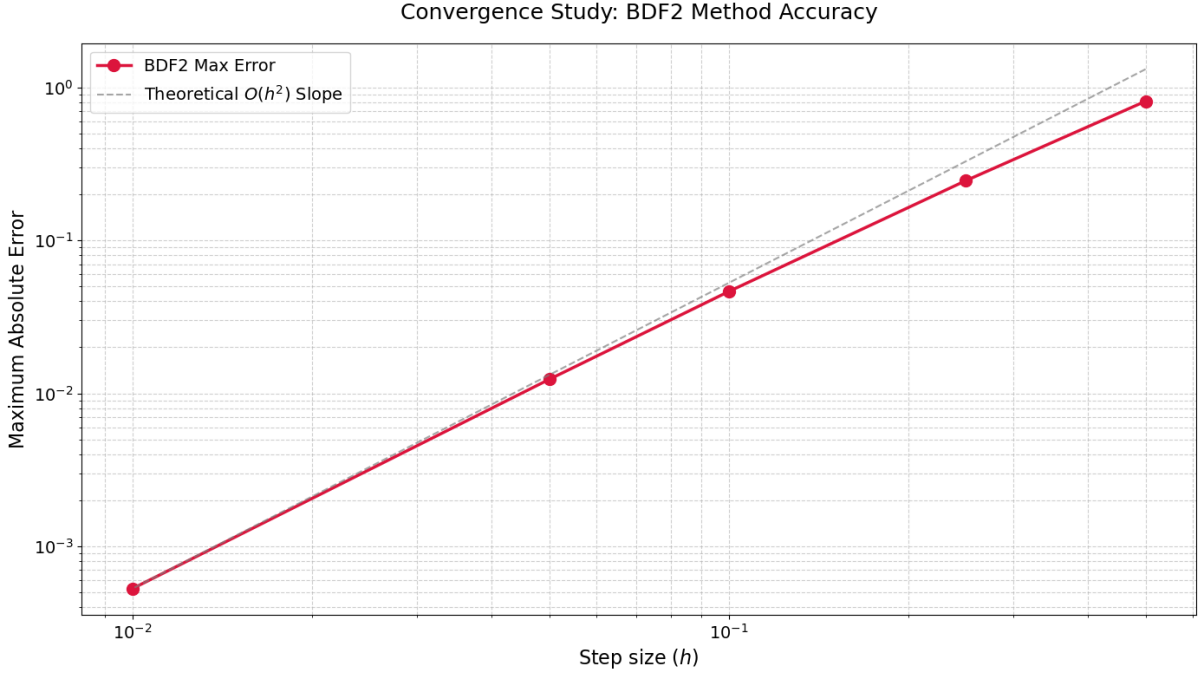


Figure 6: Log-log convergence plot for the BDF2 method. The red line represents the measured maximum absolute error, while the dashed gray line indicates the theoretical $O(h^2)$ slope.

As illustrated in Figure 6, the measured error (indicated in crimson) initially exhibits a shallower slope at larger step sizes ($h > 0.1$). This discrepancy is a common characteristic of the pre-asymptotic regime. In this region, the step size is relatively large compared to the system's time constant, meaning higher-order terms in the Taylor expansion and the first-order error introduced by the BDF1 startup step have a more pronounced influence on the total global error.

The transition toward the theoretical $O(h^2)$ slope becomes evident as the step size is refined. For values of $h < 0.1$, the error curve begins to parallel the dashed reference line more closely. In this state, the second-order truncation error of the BDF2 scheme becomes the dominant factor. This characteristic "tightening" of the slope as h decreases provides empirical evidence that the implementation is mathematically sound and that the global error will continue to diminish predictably as the temporal resolution increases.

8 Stability and Robustness in Stiff Systems

A system is classified as stiff when it contains a significant disparity between fast transient decay and slow long-term evolution. We analyze this using a relaxation model where $\lambda = 50$:

$$\frac{dy}{dt} = -50(y - 0.2) \quad (28)$$

The characteristic time scale of this system is $\tau = 1/\lambda = 0.02$. In such regimes, the choice of numerical method is not merely a matter of precision, but of existence. It determines whether

a simulation provides a valid physical result or a catastrophic numerical "blow-up."

8.1 The Stability Bottleneck of Explicit Methods

The catastrophic failure of explicit methods in stiff regimes is rooted in the mathematical properties of the "Region of Absolute Stability". To understand why these methods "blow up," we apply the explicit Euler method to the test equation $y' = -\lambda(y - y_\infty)$. The discrete update formula is given by:

$$y_{n+1} = y_n + h[-\lambda(y_n - y_\infty)] = (1 - h\lambda)y_n + h\lambda y_\infty \quad (29)$$

The term $R(h\lambda) = (1 - h\lambda)$ is defined as the amplification factor. It dictates how the solution (and any numerical error) propagates from one step to the next. For the numerical solution to remain bounded and physically consistent—specifically, to decay toward equilibrium rather than diverge—the magnitude of this factor must satisfy the stability condition:

$$|1 - h\lambda| < 1 \quad (30)$$

This inequality implies that $-1 < 1 - h\lambda < 1$.

Solving for h , we subtract 1 from all parts to get $-2 < -h\lambda < 0$, and dividing by $-\lambda$ (recalling that $\lambda > 0$) yields the stability requirement:

$$h < \frac{2}{\lambda} \quad (31)$$

In our specific model where $\lambda = 50$, this threshold is exactly $h < 0.04$.

8.1.1 Numerical Divergence at $h = 0.1$

When the step size $h = 0.1$ is used (as shown in Figure 7), the amplification factor becomes $R = 1 - (0.1)(50) = -4$. This result is significant for two reasons:

1. **Sign Oscillation:** The negative value of R causes the numerical solution to flip across the equilibrium line y_∞ at every iteration, creating the "sawtooth" behavior typical of unstable explicit solvers.
2. **Exponential Growth:** Since $|R| = 4 > 1$, the error is not merely preserved; it is quadrupled at every time step. On the logarithmic scale of Figure 7, this manifests as a linear spike, as the error grows according to the geometric progression $|E_n| \propto 4^n$.

8.1.2 Geometric Interpretation

Geometrically, this failure occurs because explicit methods are "blind" to the curvature of the solution within the interval $[t_n, t_{n+1}]$. At t_n , the solver calculates a tangent line based on an extremely steep slope. Because the step size $h = 0.1$ is large relative to the system's time constant ($\tau = 0.02$), the linear extrapolation "overshoots" the equilibrium point.

By the time the solver calculates the next slope at t_{n+1} , it finds an even steeper gradient pulling it back in the opposite direction. This creates a destructive feedback loop of over-correction. In contrast, implicit methods like BDF2 evaluate the gradient at the *future* state y_{n+1} , which naturally damps the solution and allows for stable integration even when h is significantly larger than the stability threshold of explicit schemes.

When this limit is exceeded—as demonstrated in Figure 7 with $h = 0.1$ —the explicit methods exhibit immediate exponential divergence. Each step carries the linear extrapolation far beyond

the equilibrium point y_∞ , "overshooting" the actual solution curve. Since the ODE is self-correcting, the next step calculates an even steeper slope in the opposite direction, creating a destructive feedback loop. As shown in the plot, the error for Euler and AB2 exceeds 10^3 within just five iterations.

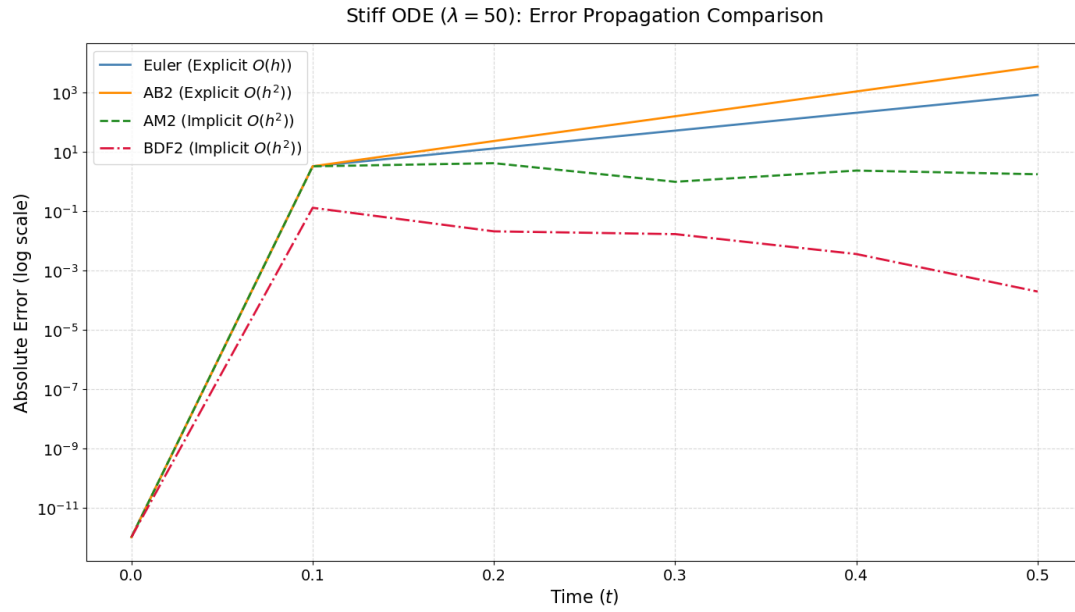


Figure 7: Log-scale absolute error for a stiff ODE ($\lambda = 50$) with $h = 0.1$.

A Two-step Adams–Bashforth method

We begin from the exact integral form of the differential equation

$$y'(t) = f(t, y(t))$$

integrated over one time step:

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt.$$

The central difficulty is that the function $f(t, y(t))$ is not known continuously on the interval $[t_n, t_{n+1}]$. What we do know are its previously computed values at earlier grid points, namely

$$f_n = f(t_n, y_n) \quad f_{n-1} = f(t_{n-1}, y_{n-1})$$

The main idea of the Adams–Bashforth family is to approximate the slope function inside the integral by an interpolation polynomial built from already known slope values. For the two-step version, we use the two most recent points

$$(t_n, f_n) \quad (t_{n-1}, f_{n-1})$$

and construct the unique linear interpolation polynomial $p_1(t)$ passing through them. Using the Lagrange interpolation formula, this polynomial is

$$p_1(t) = f_n \frac{t - t_{n-1}}{t_n - t_{n-1}} + f_{n-1} \frac{t - t_n}{t_{n-1} - t_n}$$

Since the grid is uniform with step size

$$h = t_n - t_{n-1}$$

this simplifies to

$$p_1(t) = f_n \frac{t - t_{n-1}}{h} - f_{n-1} \frac{t - t_n}{h}$$

This polynomial provides a linear approximation of how the slope evolves over time based on the recent history of the numerical solution. We now replace the exact integrand in the integral formula by this approximation:

$$\int_{t_n}^{t_{n+1}} f(t, y(t)) dt \approx \int_{t_n}^{t_{n+1}} p_1(t) dt$$

Substituting into the exact relation gives

$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} p_1(t) dt$$

Integrating for the first term,

$$\int_{t_n}^{t_{n+1}} f_n \frac{t - t_{n-1}}{h} dt = \frac{f_n}{h} \int_{t_n}^{t_{n+1}} (t - t_{n-1}) dt$$

Because

$$t_{n-1} = t_n - h$$

we have

$$t - t_{n-1} = t - t_n + h$$

Let

$$s = t - t_n$$

Then as t goes from t_n to $t_{n+1} = t_n + h$, we have

$$s \in [0, h]$$

Therefore,

$$\int_{t_n}^{t_{n+1}} (t - t_{n-1}) dt = \int_0^h (s + h) ds = \left[\frac{s^2}{2} + hs \right]_0^h = \frac{h^2}{2} + h^2 = \frac{3h^2}{2}$$

Hence,

$$\frac{f_n}{h} \cdot \frac{3h^2}{2} = \frac{3h}{2} f_n$$

For the second term,

$$\int_{t_n}^{t_{n+1}} -f_{n-1} \frac{t - t_n}{h} dt = -\frac{f_{n-1}}{h} \int_{t_n}^{t_{n+1}} (t - t_n) dt$$

Using again $s = t - t_n$,

$$\int_{t_n}^{t_{n+1}} (t - t_n) dt = \int_0^h s ds = \left[\frac{s^2}{2} \right]_0^h = \frac{h^2}{2}$$

Thus,

$$-\frac{f_{n-1}}{h} \cdot \frac{h^2}{2} = -\frac{h}{2} f_{n-1}$$

Combining both contributions gives

$$\int_{t_n}^{t_{n+1}} p_1(t) dt = \frac{3h}{2} f_n - \frac{h}{2} f_{n-1}$$

Substituting this back into the update formula yields

$$y_{n+1} = y_n + \frac{h}{2} (3f_n - f_{n-1})$$

This is the **two-step Adams–Bashforth method**.

B Accuracy and order

The order of accuracy of the two-step Adams–Bashforth method is determined by how accurately the integral

$$\int_{t_n}^{t_{n+1}} f(t, y(t)) dt$$

is approximated. We start from the exact identity

$$y(t_{n+1}) = y(t_n) + \int_{t_n}^{t_{n+1}} f(t, y(t)) dt$$

In the Adams–Bashforth 2 method, the integrand $f(t, y(t))$ is replaced by a first-degree interpolation polynomial constructed from the two previously known slope values

$$f_n = f(t_n, y_n) \quad f_{n-1} = f(t_{n-1}, y_{n-1})$$

Because this interpolation polynomial is linear, it reproduces exactly all terms up to first order in the local Taylor expansion of the slope function around t_n . To make this precise, expand the exact slope function in Taylor form:

$$f(t, y(t)) = f_n + (t - t_n) \frac{d}{dt} f(t_n, y(t_n)) + \frac{(t - t_n)^2}{2} \frac{d^2}{dt^2} f(\xi)$$

for some $\xi \in [t_{n-1}, t_{n+1}]$. The linear interpolation polynomial captures exactly the constant and linear terms:

$$f_n + (t - t_n) \frac{d}{dt} f(t_n, y(t_n))$$

The first neglected term is therefore quadratic in $(t - t_n)$, namely

$$\mathcal{O}((t - t_n)^2)$$

Since this approximation is integrated over an interval of length h , the corresponding one-step integration error becomes

$$\mathcal{O}(h^3)$$

Which is called the local truncation error. In words, this means that the error made in a single step decreases proportionally to h^3 as the step size becomes smaller. To obtain the error over the entire interval $[t_0, T]$, we must account for the accumulation of these local errors.

On a fixed interval of length

$$T - t_0$$

the number of time steps is approximately

$$N = \frac{T - t_0}{h} = \mathcal{O}(1/h)$$

Therefore, the accumulated error behaves like

$$N \cdot \mathcal{O}(h^3) = \mathcal{O}(1/h) \cdot \mathcal{O}(h^3) = \mathcal{O}(h^2)$$

This gives the global truncation error

$$e_n = \mathcal{O}(h^2)$$

Hence, the two-step Adams–Bashforth method is said to be **second-order accurate**.

This is an important improvement compared to explicit Euler, whose global error is only

$$\mathcal{O}(h)$$

In practical terms, this means that halving the step size approximately reduces the global error by a factor of

$$2^2 = 4$$

C Three-step Adams–Moulton method

The derivation once again begins from the exact integral form

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt$$

As before, the exact function $f(t, y(t))$ is not known continuously over the interval.

This time, however, we approximate it using three slope values:

$$f_{n+1}, \quad f_n, \quad f_{n-1}$$

That is, we interpolate through the three points

$$(t_{n+1}, f_{n+1}) \quad (t_n, f_n) \quad (t_{n-1}, f_{n-1})$$

Using these three points, we construct a quadratic interpolation polynomial $p_2(t)$ which approximates the slope over the interval. Since three points are used, the polynomial is of degree two, which allows it to capture curvature in the slope evolution.

Interpolation idea

Using the Lagrange form, the polynomial is written as

$$p_2(t) = f_{n+1}L_{n+1}(t) + f_nL_n(t) + f_{n-1}L_{n-1}(t)$$

where $L_j(t)$ are the quadratic Lagrange basis polynomials. Rather than writing the full basis functions explicitly, the important idea is that this polynomial satisfies

$$p_2(t_{n+1}) = f_{n+1}$$

$$p_2(t_n) = f_n$$

$$p_2(t_{n-1}) = f_{n-1}$$

This means the slope is approximated by a quadratic curve passing through the most recent two known slopes and the unknown future slope.

Integration of the interpolation polynomial

We begin from the exact integral formulation of the differential equation:

$$y(t_{n+1}) - y(t_n) = \int_{t_n}^{t_{n+1}} f(t, y(t)) dt$$

In the Adams–Moulton approach, we approximate the integrand by the quadratic interpolation polynomial:

$$f(t, y(t)) \approx p_2(t)$$

This leads to the approximation

$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} p_2(t) dt$$

To compute this integral, it is convenient to shift the time interval to a local coordinate system. Let

$$s = \frac{t - t_n}{h} \quad t = t_n + hs$$

so that:

$$t_n \rightarrow s = 0 \quad t_{n+1} \rightarrow s = 1 \quad t_{n-1} \rightarrow s = -1$$

In this scaled variable, the integral becomes

$$\int_{t_n}^{t_{n+1}} p_2(t) dt = h \int_0^1 \tilde{p}_2(s) ds$$

where $\tilde{p}_2(s)$ is the interpolation polynomial written in the variable s . Instead of writing the full Lagrange basis functions explicitly, we use the known result that the integral of a quadratic interpolation polynomial over equally spaced nodes $s = -1, 0, 1$ gives fixed weights for each function value.

Thus we write

$$\int_0^1 \tilde{p}_2(s) ds = af_{n+1} + bf_n + cf_{n-1}$$

The coefficients are obtained by integrating the corresponding Lagrange basis polynomials over $[0, 1]$.

On the interval $[t_{n-1}, t_n, t_{n+1}]$ we introduce the scaled variable

$$s = \frac{t - t_n}{h}$$

so that the nodes become

$$s_{n-1} = -1 \quad s_n = 0 \quad s_{n+1} = 1$$

We now define the Lagrange basis polynomials in the variable s .

Basis function for f_{n+1}

$$L_{n+1}(s) = \frac{(s-0)(s+1)}{(1-0)(1+1)} = \frac{s(s+1)}{2}$$

We compute its contribution:

$$a = \int_0^1 L_{n+1}(s) ds = \int_0^1 \frac{s(s+1)}{2} ds$$

Expanding:

$$\frac{s(s+1)}{2} = \frac{s^2 + s}{2}$$

Thus:

$$a = \frac{1}{2} \int_0^1 (s^2 + s) ds = \frac{1}{2} \left[\frac{s^3}{3} + \frac{s^2}{2} \right]_0^1$$

Evaluating:

$$a = \frac{1}{2} \left(\frac{1}{3} + \frac{1}{2} \right) = \frac{1}{2} \cdot \frac{5}{6} = \frac{5}{12}$$

Basis function for f_n

$$L_n(s) = \frac{(s+1)(s-1)}{(0+1)(0-1)} = -(s+1)(s-1) = 1 - s^2$$

Now:

$$b = \int_0^1 (1 - s^2) ds = \left[s - \frac{s^3}{3} \right]_0^1$$

Evaluating:

$$b = 1 - \frac{1}{3} = \frac{2}{3}$$

Basis function for f_{n-1}

$$L_{n-1}(s) = \frac{s(s-0)}{(-1-0)(-1-1)} = \frac{s(s-0)}{(-1)(-2)} = \frac{s(s-1)}{2}$$

Thus:

$$c = \int_0^1 \frac{s(s-1)}{2} ds = \frac{1}{2} \int_0^1 (s^2 - s) ds$$

Compute:

$$c = \frac{1}{2} \left[\frac{s^3}{3} - \frac{s^2}{2} \right]_0^1$$

Evaluating:

$$c = \frac{1}{2} \left(\frac{1}{3} - \frac{1}{2} \right) = \frac{1}{2} \cdot \left(-\frac{1}{6} \right) = -\frac{1}{12}$$

Result

Thus the coefficients are

$$a = \frac{5}{12}, \quad b = \frac{2}{3}, \quad c = -\frac{1}{12}$$

Therefore,

$$\int_{t_n}^{t_{n+1}} p_2(t) dt = h \left(\frac{5}{12} f_{n+1} + \frac{2}{3} f_n - \frac{1}{12} f_{n-1} \right)$$

Substituting this into the integral equation gives

$$y_{n+1} = y_n + h \left(\frac{5}{12} f_{n+1} + \frac{2}{3} f_n - \frac{1}{12} f_{n-1} \right)$$

This is the Adams–Moulton method, which is an implicit method because the term $f_{n+1} = f(t_{n+1}, y_{n+1})$ depends on the unknown future value y_{n+1} .

D Construction of the derivative approximation - BDF2

We assume a uniform step size h , so that the three time levels can be written relative to the newest point t_{n+1} as

$$t_n = t_{n+1} - h \quad t_{n-1} = t_{n+1} - 2h$$

The goal is to approximate the unknown derivative $y'(t_{n+1})$ using only discrete solution values. The key idea is to replace the unknown smooth function $y(t)$ locally by a simpler function that is fully determined by known data points. We construct the unique quadratic interpolating polynomial $p_2(t)$ that satisfies

$$p_2(t_{n+1}) = y_{n+1} \quad p_2(t_n) = y_n \quad p_2(t_{n-1}) = y_{n-1}$$

This polynomial can be interpreted as the simplest smooth curve that exactly passes through the three most recent solution values. The assumption behind this construction is that, on a sufficiently small time interval, the true solution behaves similarly to this quadratic approximation.

Instead of using the polynomial itself, we are interested in its rate of change at the newest time level. Since $p_2(t)$ is explicitly known in terms of its interpolation points, we can differentiate it analytically and evaluate the result at t_{n+1} .

Carrying out this differentiation leads to a weighted combination of the three solution values:

$$p_2'(t_{n+1}) = \frac{3y_{n+1} - 4y_n + y_{n-1}}{2h}$$

This expression is obtained by constructing the quadratic interpolating polynomial through the three points

$$(t_{n+1}, y_{n+1}), \quad (t_n, y_n) \quad (t_{n-1}, y_{n-1})$$

and differentiating it at t_{n+1} . We introduce a shifted variable

$$s = t - t_{n+1}$$

so that the interpolation nodes become

$$s_0 = 0 \quad s_1 = -h \quad s_2 = -2h$$

In Lagrange form, the interpolating polynomial is

$$p_2(t) = y_{n+1}L_0(s) + y_nL_1(s) + y_{n-1}L_2(s)$$

where the basis polynomials are

$$L_0(s) = \frac{(s+h)(s+2h)}{2h^2} \quad L_1(s) = \frac{s(s+2h)}{-h^2} \quad L_2(s) = \frac{s(s+h)}{2h^2}$$

We now differentiate and evaluate at $s = 0$ (which corresponds to $t = t_{n+1}$):

$$L'_0(0) = \frac{3}{2h} \quad L'_1(0) = -\frac{4}{2h} \quad L'_2(0) = \frac{1}{2h}$$

Therefore,

$$p'_2(t_{n+1}) = y_{n+1}L'_0(0) + y_nL'_1(0) + y_{n-1}L'_2(0)$$

which gives

$$p'_2(t_{n+1}) = \frac{3y_{n+1} - 4y_n + y_{n-1}}{2h}$$

This formula can be understood as a refined finite difference approximation of the derivative. Unlike the simple backward difference used in BDF1, this expression incorporates curvature information through the second previous point y_{n-1} , which improves accuracy.

The coefficients 3, -4 , and 1 arise directly from the structure of the quadratic interpolation: they ensure that the polynomial matches all three data points exactly, and that its derivative is consistent with this reconstruction at t_{n+1} .

In this way, the derivative approximation is not chosen heuristically, but is fully determined by enforcing that a quadratic function matches the known solution history and is then differentiated exactly.